Tekst do powiedzenia

Po polsku. Numery odpowiadają stronom sprawozdania; nie trzeba czytać kodu linia po linii.

Strona 1 - Binet

Porównaliśmy Bineta, Strassena i AlphaTensor. Mierzyliśmy czas i liczbę operacji, a wyniki sprawdzaliśmy w Octave. Binet dzieli macierze na cztery bloki. Każdy blok wyniku jest sumą dwóch mniejszych iloczynów, więc wykonujemy osiem mnożeń rekurencyjnych.

Strona 2 - Dowolne rozmiary Bineta

Najpierw rozważaliśmy dopisywanie zer do najbliższej potęgi dwójki. To jednak mogłoby mocno zwiększyć rozmiar tablic i zużycie pamięci. Dlatego dzielimy wymiary nierówno: na przykład pięć na dwa i trzy. Wspólny wymiar obu macierzy dzielimy w tym samym miejscu. Gdy któryś wymiar wynosi jeden, mnożymy bezpośrednio.

Strona 3 - Liczniki i czas

Każde wywołanie zwraca wynik oraz liczbę mnożeń i dodawań. Sumujemy liczniki i doliczamy dodawanie elementów bloków. Mierzyliśmy wybrane rozmiary do 512: dwa razy do 128 i po razie dla 256 oraz 512. Binet dla 512 potrzebował około 427 sekund.

Strona 4 - Przejście do Strassena

Binet ma złożoność sześcienną. Strassen ogranicza liczbę mnożeń z ośmiu do siedmiu, ale wymaga dodatkowych sum i różnic bloków. Dlatego mniej mnożeń nie oznacza automatycznie mniejszej sumy wszystkich operacji.

Strona 5 - Siedem iloczynów

Tworzymy kombinacje bloków wejściowych i liczymy siedem iloczynów. Z nich składamy cztery części wyniku. Każde mnożenie wykonujemy ponownie Strassenem. Dodawania bloków też zliczamy: jedna suma bloków oznacza wiele dodawań liczb.

Strona 6 - StrassenOdd

Nierówne bloki nie pasują tutaj, bo Strassen dodaje je do siebie przed mnożeniem. Dla nieparzystego rozmiaru oddzielamy ostatni wiersz i kolumnę. Część główną mnożymy Strassenem, pozostałe iloczyny zwykłą metodą i składamy wynik. Tak obsługujemy dowolne dodatnie rozmiary kwadratowe bez dopisywania zer.

Tekst do powiedzenia - ciąg dalszy

Krótko o wynikach i problemach.

Strona 7 - Wyniki Strassena

Strassen ma złożoność około n do potęgi 2,81. Dla dużych macierzy był szybszy od Bineta. Jego zaletą jest mniej mnożeń, ale dodatkowe dodawania powodują, że suma FLOP-ów nie zawsze jest mniejsza.

Strona 8 - AlphaTensor

Tutaj korzystamy z gotowych współczynników opublikowanych przez DeepMind. Nie trenujemy modelu. Dzielimy A na cztery na pięć bloków, a B na pięć na pięć. Kombinacje bloków pozwalają wykonać 76 mnożeń zamiast stu. W tej metodzie dopisujemy zera do wymaganych wielokrotności i na końcu przycinamy wynik.

Strona 9 - Skoki czasu

AlphaTensor dla 256 potrzebował około dziesięciu sekund, a dla 512 około 284 sekund. Koszt rośnie nierówno, bo zmienia się dopełnienie zerami i liczba poziomów rekurencji. Dlatego nie był najszybszy dla każdego rozmiaru.

Strona 10 - Porównanie

Osie są logarytmiczne, więc wzrost potęgowy może wyglądać jak prosta. Binet ma złożoność sześcienną, Strassen około n do potęgi 2,81. W AlphaTensor koszt zależy od nierównego podziału i dopełnienia. Sama liczba mnożeń nie wystarcza do przewidzenia czasu działania.

Strona 11 - Poprawność

Porównywaliśmy wyniki z Octave na tych samych danych. Dopuszczamy małe różnice, bo kolejność działań wpływa na zaokrąglenia. Wszystkie trzy algorytmy przeszły sprawdzenie z tolerancją bezwzględną i względną. Najszybsza metoda zależała od rozmiaru macierzy.

Jeśli padnie pytanie o Cython

Kod algorytmów przetłumaczyliśmy na C i zbudowaliśmy moduły natywne. Uruchamiamy je z Pythona. Nie dodawaliśmy wszędzie typów C, więc samo skompilowanie nie daje dużego przyspieszenia.

Для себя: в таблице Страссена в PDF остались старые времена. Новый замер Cython для 512: 197,7 с. Не повторяй неподтверждённую степень сложности AlphaTensor из отчёта.

Бинет: что нарисовать на доске

Пример 5×5. Здесь блоки могут быть разного размера.

1. Нарисуй A и B. В каждой проведи линии после второй строки и второго столбца. Получится деление 5 = 2 + 3.

A11 2×2	**A12** 2×3
A21 3×2	**A22** 3×3

B11 2×2	**B12** 2×3
B21 3×2	**B22** 3×3

2. Покажи один блок результата. В обоих слагаемых итоговый размер одинаковый, поэтому их можно сложить:

C11 = A11·B11 + A12·B21

(2×2)·(2×2) + (2×3)·(3×2) = (2×2) + (2×2)

Остальные блоки считаются так же. Каждое умножение снова вызывает Бинета. Никаких нулей не добавляем.

Как обрабатываем прямоугольные матрицы

Для A размера r×k и B размера k×s отдельно делим r, k и s пополам с округлением вниз. Остаток попадает во вторую часть. Колонки A и строки B обязательно делим по одной границе k // 2.

Когда останавливаемся

Если r = 1, k = 1 или s = 1, считаем обычными циклами. Это важно: условие не только «матрица 1×1». Оно покрывает строку, столбец и другие узкие блоки.

Почему сначала думали о нулях

Если расширять каждую матрицу 5×5 до 8×8, вместо 25 элементов в одном буфере будет 64. Для float64 это 200 и 512 байт. Это сравнение размеров буферов, а не точное сравнение пиков памяти всего алгоритма. Поэтому отказались от дополнения до степени двойки.

Страссен: что нарисовать на доске

Пример 5×5. Основная часть 4×4 плюс край.

1. Нарисуй A и B. Теперь линии проходят перед последней строкой и последним столбцом, то есть делим 5 = 4 + 1.

A0 4×4	**a** 4×1
r 1×4	**α** 1×1

B0 4×4	**b** 4×1
s 1×4	**β** 1×1

2. Обведи A0 и B0: только их произведение здесь считаем Страссеном. Серые края участвуют в обычном умножении.

C11 = Strassen(A0, B0) + a·s → 4×4

C12 = A0·b + a·β → 4×1

C21 = r·B0 + α·s → 1×4

C22 = r·b + α·β → 1×1

Важная деталь: a·s - столбец на строку, получается целая матрица 4×4. А r·b - строка на столбец, получается одно число. Это разные операции.

Почему это работает для любого квадратного размера

n = 1: одно умножение. n чётное: четыре равных блока и семь рекурсивных произведений. n нечётное: убираем край, считаем часть n−1 и возвращаем вклад края по формулам выше.

Например, 7×7 → основная часть 6×6 → блоки 3×3 → основная часть 2×2 → 1×1. Проверка чётности выполняется заново при каждом вызове.

Что сказать: мы не выбрасываем последний ряд и столбец. Все их вклады учтены. Блоки 2×3 и 3×2, как у Бинета, здесь не подходят: в формулах Страссена нужно складывать равные по форме блоки.

AlphaTensor: как обрабатываем размеры

Короткая схема, если спросят про третий алгоритм.

Здесь дополнение нулями всё-таки есть

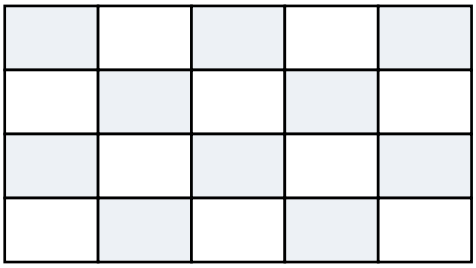
У Бинета - неравные блоки; у Страссена - отделение края; у AlphaTensor - дополнение до ближайших нужных кратностей. Это три разных способа.

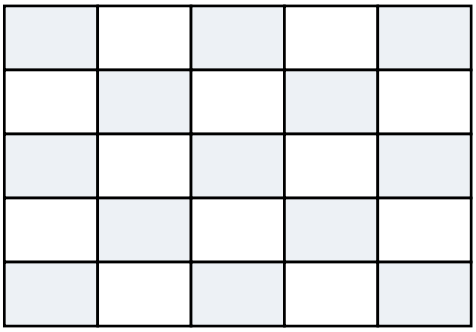
Пусть A имеет размер 7×11, а B - 11×9. Строки A округляем вверх до кратного 4, общий размер и колонки B - до кратного 5:

A: 7×11 → 8×15

B: 11×9 → 15×10

Дальше A делим на сетку из 4 строк и 5 столбцов блоков, B - из 5 строк и 5 столбцов блоков. На доске достаточно нарисовать две такие сетки.





Блок A: 2×3. Блок B: 3×2. Блок результата: 2×2.

Что происходит дальше

По таблицам коэффициентов собираем 76 пар комбинаций блоков и рекурсивно перемножаем их. Затем из этих произведений собираем результат 8×10 и берём только исходные 7×9.

В этом примере произведения имеют форму (2×3)·(3×2). Так как число строк меньше четырёх, рекурсия останавливается и используется обычное умножение.

Общее правило остановки

Если строк A меньше 4, общий размер меньше 5 или колонок B меньше 5 - обычное умножение. Иначе снова дополнение, разбиение и 76 произведений. Арифметика с добавленными нулями тоже входит в счётчики.